

第 21 届全国大学生 智能汽车竞赛 技 术 报 告

学 校:	哈尔滨工程大学
队 伍 名 称:	济海追风 8 队
参 赛 队 员:	郭 赫 严 语 赵均昊 谢长佐
带 队 教 师:	张立伟

关于技术报告和论文使用授权的说明

本人完全了解全国大学生智能汽车竞赛有关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：

郭赫

李语

朱均昊

谢长佐

带队教师签名：

张王伟

日期：2026年8月19日

主要符号表

符号/缩略词	含义
TC387QP	英飞凌 AURIX TC387QP 微控制器平台
PWM	脉宽调制，用于电机驱动和转向执行器控制
PID	比例-积分-微分控制器
IMU	惯性测量单元
DMA	直接存储器访问
DSRAM	数据存储区域，工程中用于多核图像槽位
Otsu	大津法，用于图像阈值计算
SG 滤波	Savitzky-Golay 路径平滑滤波
AE	自动曝光（Auto Exposure）
FPS	每秒帧数
PIT	周期中断定时器

摘要

本报告介绍一套基于英飞凌 AURIX TC387QP 的智能车控制方案。工程采用模块化软件结构，将设备驱动、车辆状态、路径与控制算法、摄像头识别以及多核任务调度分层组织。源码中已实现电机增量式编码器测速、角度编码器转角测量、IMU 数据采集与 Mahony 姿态融合、里程计位置积分、摄像头图像处理、双圆目标识别与跟随、速度/角度 PID、轨迹回放及可选前视点选择等功能。摄像头新跟随路径采用 CPU0 负责 DMA 采集与结果处理、CPU1 负责图像处理的流水线结构，并通过固定槽位、邮箱和快照机制管理图像所有权。

关键词：TC387QP；多核流水线；双圆目标识别；PID 控制；Mahony 姿态融合；轨迹回放；智能车

第一章 绪论

1.1 项目背景

智能车竞赛要求参赛车辆在限定场地和规则约束下完成感知、定位、决策与运动控制。技术报告的作用是说明方案如何从机械结构、硬件接口、底层驱动、感知算法和控制算法形成闭环，并大致验证方案的有效性。

本项目以 TC387 逐飞科技开源工程为软件基础，面向带有摄像头目标识别、路径记录/回放和车辆闭环控制需求的智能车系统。报告重点放在工程中已经出现的技术实现，讲解技术思路。

1.2 设计目标

系统设计目标包括：完成车辆底盘运动控制；稳定获取轮速、舵角、姿态、里程和位置信息；完成摄像头图像采集、目标识别与图像解析；支持相机跟随、遥控器控制、轨迹记录与轨迹回放等工作模式；在多核环境下避免 DMA、图像算法和显示之间的内存冲突；提供参数存储、菜单调试和故障停机接口。

1.3 报告范围

本报告覆盖总体方案、机械与硬件接口、软件架构、感知与控制算法。

第二章 系统总体方案

2.1 系统组成

系统由车辆机械本体、TC387QP 主控、逐飞科技总钻风摄像头、左右轮增量式编码器、前轮角度编码器、IMU、IPS200 显示与按键互模块、无线/语音/灯板等外设组成。主控读取传感器数据，经过图像识别、姿态估计、位置更新和控制算法，输出左右驱动电机 PWM 以及前轮转向电机控制信号。

表 2.1 系统分层与模块

层级	主要模块	源码依据/说明
感知层	MT9V03X 摄像头（总钻风）、增量式编码器、角度编码器、IMU	Camera_New、encoder、IMU 等文件
状态层	车速、里程、航向角、二维位置、车辆状态标志	my_car、Position 等文件
算法层	Otsu/连通域/双圆识别、PID、Mahony、SG 滤波、前视回放	Camera_New、PID、IMU、SG_Filter、Lookahead 等文件
执行层	北京科宇 T 型卡丁车原装左右轮电机/转向电机、IPS200、无源 3W 喇叭（内阻 4R）、CI1302 语音芯片、英飞凌 TLD7002 灯板	motor、buzzer、Voice、TLD7002 等文件

2.2 多核任务与数据流

源码将 CPU0、CPU1、CPU2、CPU3 入口分别保留。当前工程中，CPU0 负责系统启动、摄像头流水线启动、车辆初始化和菜单、车辆控制；CPU1 负责 Camera_New_Worker 图像处理服务；CPU2 与 CPU3 入口保留为空循环。

新摄像头流水线的核心数据流为：摄像头 DMA 写入 CPU1 DSRAM 的图像槽位；DMA 完成中断发布已完成槽位；CPU1 获取最新槽位并执行曝光统计和双圆识别；CPU1 将结果写入结果邮箱，并按较低频率发布显示快照；CPU0 读取稳定结果、进行跟随控制和显示。槽位状态包括 FREE、DMA_WRITING、PENDING_LATEST、CPU1_PROCESSING，体现了明确的图像所有权。

2.3 工作模式

表 2.2 工作模式

模式	主要任务	待补充内容
普通车辆控制	编码器/IMU 采集、位置更新、电机和转向输出	通过遥控器控制
摄像头跟随	双圆目标识别、横向/距离误差计算、角度与速度控制	在特殊目录下先进行图像基础标定，在标定基础上进行图像解算、车辆控制等任务
轨迹记录	记录位置点、特殊点标志和速度信息	在车辆行驶过程中实时解算
轨迹回放	按回放路径计算目标角度、速度和停止/倒车逻辑	各科目路径复现采用不同 PID 参数等等
标定与调试	相机灰度/几何标定、舵机中值、屏幕菜单和参数读写	通过 IPS200 屏幕显示、按键调整参数并存入 FLASH

2.4 故障与安全策略

摄像头跟随代码包含目标丢失、控制停止、工作线程握手超时、结果过期和流水线统计等状态。出现控制停止条件时，代码将速度期望、转向期望以及图像控制标志清零。对于 DMA 与 CPU1 处理之间的竞争，流水线采用非阻塞锁和槽位释放策略，避免在中断中长时间等待。整车控制中额外增加遥控器紧急停车功能，预防突发状况。

第三章 机械结构与安装

3.1 底盘几何参数

表 3.1 车辆几何与里程换算源码配置（所有数据皆为估计值，未精确测量）

参数	源码配置值	单位/备注
驱动轮半径 car_R	0.125	m；需用实测轮胎有效半径复核

参数	源码配置值	单位/备注
轴距 car_L	0.635	m；需核对安装中心距
轮距 car_D	0.8	m；源码注释说明为调整后的值
倒车轮距 car_D_Back	0.68	m；用于倒车转向几何
转向角上限	± 30.0	deg；由 MOTOR_DUOJL_ANGLE_MAX 限制
编码器每轮转计数	2321	count/rev；由 sum_encoder_rad 配置

3.2 机械安装要点

摄像头应保持固定，镜头光轴和车体中心线的相对位置需要记录；角度编码器中值应在车轮机械中位时重新标定；左右增量式编码器方向应通过低速前进和倒车分别验证；电机、舵机和线束安装应避免运动干涉。

3.3 机械误差与补偿

轮径差、左右增量式编码器方向、转向零位、车轮打滑和摄像头安装倾角都会影响位置和跟随控制。工程中通过左右轮 PID、角度编码器中值参数、IMU 零偏采样、相机几何标定以及里程计中点航向积分降低影响。

第四章 硬件系统设计

4.1 电路系统总体设计

整车电路系统由主驱一体主控板、离线语音模块、TLD7002 灯板驱动及硅麦放大模块的自制 PCB 组成，均使用嘉立创 EDA 专业版设计。辅助板包括电机端保护板、摄像头转接板。备赛前期采用分板设计，但限制于车的尺寸，走线

比较困难。经过团队设计，采用主驱一体板，外设分板，方便走线同时也保证了电路系统的稳定。

表 4.1 电路系统总体设计

电路板	尺寸	层数	板厚	主要功能
主驱一体主控板	99 mm×99mm	6 层	1.6 mm	主控、供电、三路有刷电机驱动及各 外设接口
离线语音模块	30mm×53mm	2 层	1.6 mm	CI1302 为核心，进行语音处理后通过 串口传输信号到单片机
灯板驱动	45mm×44mm	4 层	1.6 mm	采用英飞凌的 TLD7002 驱动 3 个 5*7 的 LED 点阵屏
硅麦放大模块	13mm×19mm	2 层	1.6 mm	对输入语音信号进行放便于后续处 理

4.2 传感器与执行器

表 4.2 传感器与执行器

部件	功能	接口/源码说明
MT9V03X 摄像头	灰度图像采集、双圆目标识别	UART/DMA/VSYNC；新路径支持多核流水线
左右轮编码器	轮速和里程测量	TIM4/TIM2 编码器接口；代码对右侧计数取反
前轮角度编码器	转向角反馈	SPI4；角度范围在代码中限制为±30°
IMU	角速度、加速度和航向估计	imu963ra 接口；初始化阶段采样 1000 次估计零偏
驱动电机	车辆前进/倒车	20 kHz PWM；方向 GPIO 与

部件	功能	接口/源码说明
		PWM 分离
转向电机/舵机机构	前轮转角控制	20 kHz PWM；带角度死区和输出斜率限制
IPS200/TFT	图像、状态和参数显示	SPI 显示接口；跟随调试界面按周期刷新
蜂鸣器、语音、灯板、遥控	提示、交互和辅助执行	对应 buzzer、Voice、TLD7002、remote 模块

4.3 主控与外设接口

主控电路采用 TC387 微控制器，连接陀螺仪、编码器、摄像头、屏幕、离线语音模块、灯板及辅助接口。

表 4.3 关键 IO 分配

功能	引脚/接口（源码配置）	备注
左轮方向/PWM	P02_6 / ATOM0_CH7_P02_7	car_params.h
右轮方向/PWM	P02_4 / ATOM0_CH5_P02_5	car_params.h
转向方向/PWM	P21_3 / ATOM0_CH0_P21_2	car_params.h
左轮编码器	TIM4: P02_8、P00_9	双通道正交输入
右轮编码器	TIM2: P33_7、P33_6	双通道正交输入
绝对值编码器	SPI4: P22_0、P22_3、P22_1，CS=P23_1	转向角反馈
IMU	SPI0: P20_11、P20_14、P20_12、P20_13	源码推荐 IO 分配
显示	SPI2: P15_3、P15_5、P15_4、P15_2、P15_1、P15_0	屏幕接口和控制信号

4.4 供电部分电路

整车采用 24V 铅酸蓄电池供电，经过电源防过压电路接到开关。主板设置电池输入与多路电源变换电路：SP1N28STER 用于 5 V 降压供电以及输出 6-10V 可调电压对灯板进行单独供电，RT9013 用于 3.3 V 电源稳压，摄像头采用 RT9013 单独供电，保证摄像头电源的稳定。在 24 V 输入条件下，整车静止待机电流约 50 mA，对应输入功耗约 1.2 W；系统运行时输入电流约 8A，对应输入功耗约 192 W。上述两项功耗按输入电压与电流乘积换算。该值不是电流表直接测量结果。

4.5 驱动部分电路

行进系统采用北京科宇的 KW-068 有刷电机，电机电压 24V，空载转速 500RPM $\pm$ 10%。转向系统采用北京科宇的有刷电机。主板设置三路 DRV8701E 栅极驱动器，并配合 AOD4184 功率 MOSFET 构成两组有刷电机全桥驱动电路；SN74HC125A 用于相关使能信号处理。

4.6 离线语音模块电路（CI1302）

基于 CI1302 离线语音方案，板载功放电路与 MIC 接口。MIC 信号经 C13 耦合电容接入功放，功放输出经 100 μ F 电容滤波后由 MIC OUT 输出，该反馈信号接入主控 AEC 引脚用于回声消除。MIC 接口部分，板载 MIC1~MIC608 共 608 路麦克风输入接口，按项目实际配置接入单路或多路麦克风。整板由 5V 供电，主控内置 PMU 电源管理单元，可支持本地语音唤醒、命令词识别及语音播报反馈功能。

4.7 TLD7002 灯板驱动电路

板载 TLD7002-16ES 多通道 LED 驱动芯片，该芯片是一款 16 通道的汽车级恒流源驱动器，各通道电流（6 位分辨率）及 PWM 占空比（14 位分辨率）均可独立配置，单通道最大输出电流为 76.5mA，输出级可并联以驱动更大电流负载。芯片通过 HSLI（高速照明接口）进行通信控制，兼容 UART over CAN 协议，最高通信速率可达 2Mbit/s。

本方案中，TLD7002-16ES 用于驱动 13 个 5 $\times$ 7 点阵 LED 灯板。每个 5 $\times$ 7 灯板共需控制 35 个 LED 点阵像素，13 个灯板合计需驱动 455 个 LED 像素点。芯片

的 16 个通道需通过通道复用与 PWM 扫描控制的方式，分时驱动各灯板的行/列电极，以实现全灯板点亮。TLD7002-16ES 支持 100Hz~2kHz 的独立 PWM 频率配置，可满足点阵扫描的动态显示需求。

4.8 电机端保护板、摄像头转接板及其它辅助电路

鉴于电机强大的反电动势，设计了电机端保护板。电机端保护板并联了 TVS 管和一个 100NF 陶瓷电容。TVS 管负责吸收线路寄生电感产生的瞬间高压尖峰，陶瓷电容处理了剩下的 10% 高压尖峰和电磁干扰。

摄像头转接板将 FPC 排线接口转换为 2.54 的排线接口，保证这个系统连接线的稳定。

4.9 PCB 设计与可靠性考虑

主驱一体主控板原理图和 PCB 由本团队使用嘉立创 EDA 自主设计。PCB 尺寸为 99mm×99 mm，采用六层结构和 1.6 mm 板厚；嘉立创 EDA 工程中实际启用 Top Layer、Bottom Layer、GND1、S1、POWER、GND2，共六个信号层。对最终 PCB 中的图层、走线和覆铜网络进行只读复核后，可得到表 4.4 所示的板层用途。

表 4.4 主板六层结构及主要用途

层序	嘉立创 EDA 层名	经 PCB 数据确认的主要用途
L1	Top Layer	顶层器件贴装、信号布线及局部电源/地覆铜
L2	GND1	PGND 整面覆铜
L3	S1	信号布线
L4	POWER	电源覆铜
L5	GND2	GND 整面覆铜,作为主要参考地平面
L6	Bottom Layer	底层器件贴装、信号与功率布线及局部电源/地覆铜

器件采用双面贴装。底层以 TC387 主控、电源接口、外设接口、LED 指示灯以及需要大面积散热的 MOSFET 为主；顶层集中布置电源部分电路、两片 DRV8701E 有刷驱动。结合原理图功能和 PCB 器件位置，主板可划分为主控与数字接口区、电源变换区、有刷驱动区。该划分描述的是主要功能聚集关系，各区域在紧凑板面内并非完全物理隔离。

PCB 中同时设置 GND 与 PGND 网络；GND2 提供连续 GND 参考面，POWER 承载多路电源覆铜。以上内容均来自最终 PCB 图元与网络数据，未据此推断未经生产文件证明的介质厚度、铜厚或阻抗参数。

图 4.1 和图 4.2 展示了焊接前 PCB 的正反面，可用于核对板形、器件焊盘、接口位置、功率走线和队伍标识；焊接完成后的主板安装状态可参见图 4.3 和图 4.4。

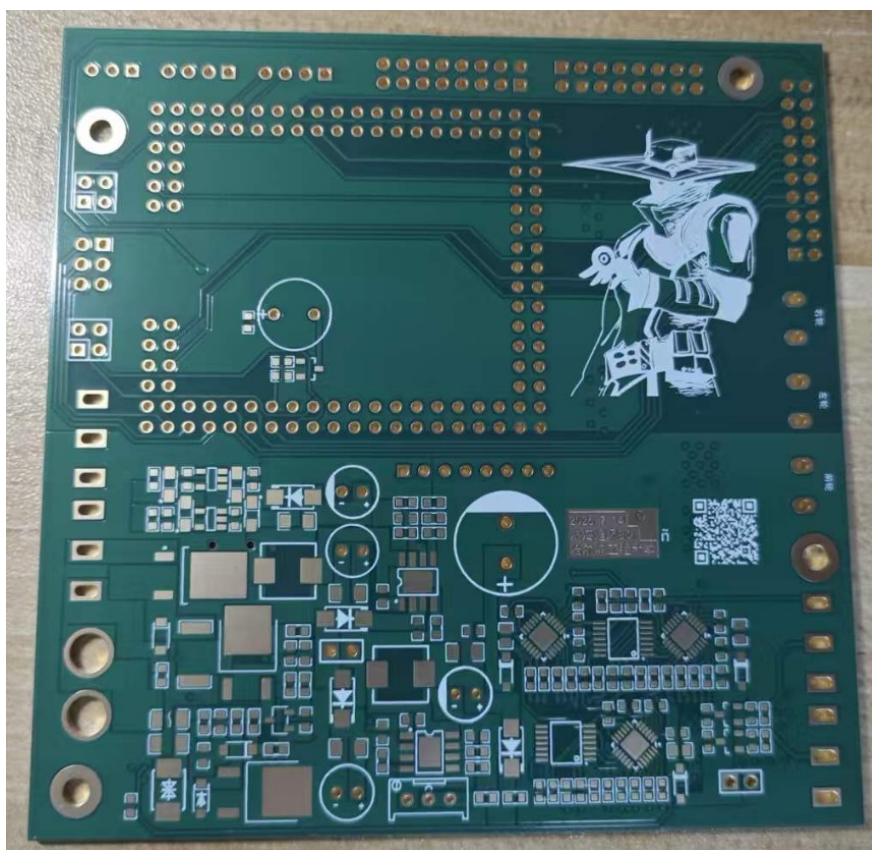


图 4.1 未焊接 PCB 正面

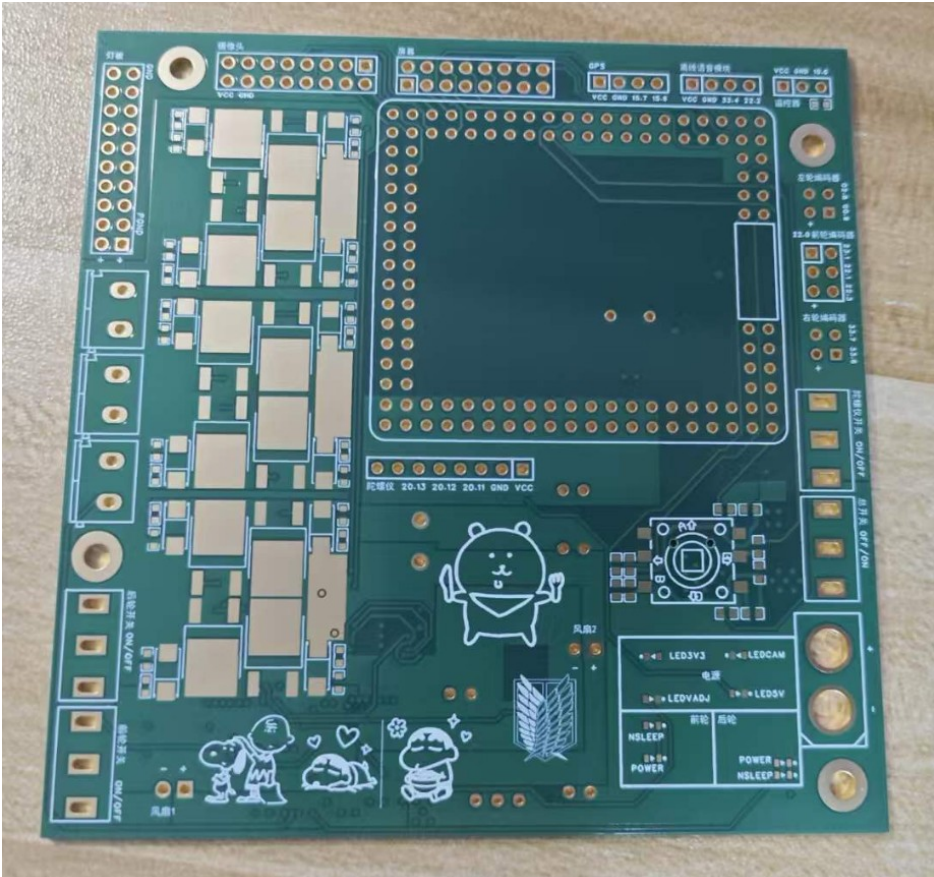


图 4.2 未焊接 PCB 反面



图 4.3 已焊接 PCB 反面

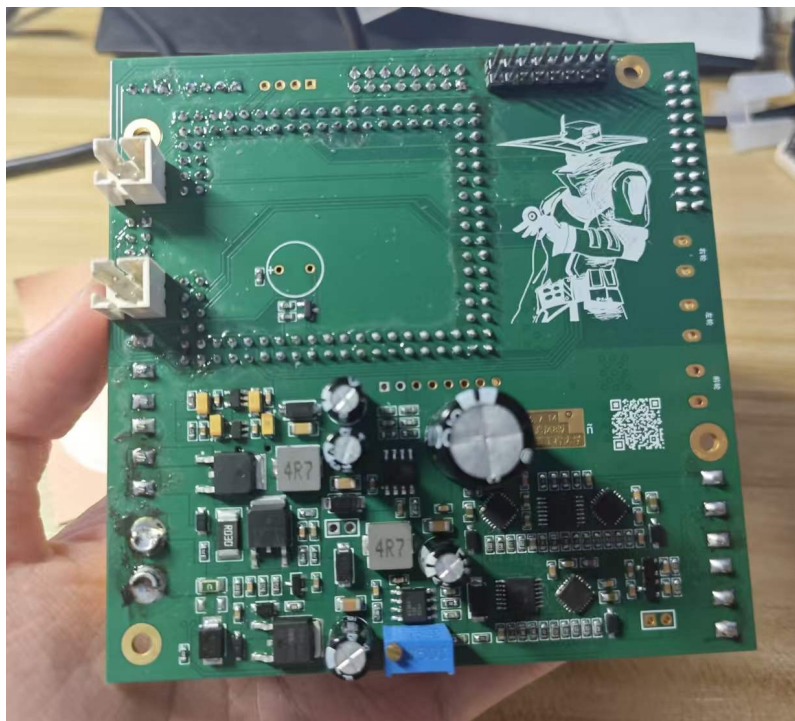


图 4.4 已焊接 PCB 正面

第五章 软件系统设计

5.1 软件分层与文件组织

表 5.1 软件分层与文件组织

目录/模块	职责	代表文件
Car_Device	外设驱动、传感器接口、TC387 内部存储器等	Camera、Camera_New（更新后的摄像头处理）、motor、encoder、IMU、Flash、remote 等
Car_arithmetic	车辆状态、控制和路径算法	PID、Position、Trace、Lookahead、SG_Filter、Subject_One~Four 等
user	多核入口和中断服务	cpu0_main~cpu3_main、isr、isr_config
libraries	Seekfree 底层库和器件驱动	由逐飞科技开源工程提供

5.2 初始化与周期任务

车辆初始化函数 my_car_init() 依次初始化按键、PID、IPS200 屏幕显示、Flash 参数、无源 3W 喇叭、IMU、增量式编码器、角度编码器、遥控器、北京科宇原装电机和 CI1302 语音模块，并启动四路中断。CCU60/CCU61 的三个通道采用 5 ms 周期，另一路采用 10 ms 周期；其中 5 ms 周期用于位置/电机输出和回放角度更新，另一个 5 ms 周期用于摄像头控制量更新。IMU 头文件将 Mahony 采样频率配置为 200 Hz。

表 5.2 初始化与周期任务

周期/频率	代码中的用途	性质
5 ms	按键/遥控/舵角采样；位置更新、电机输出、回放角度配置	源码调度配置

周期/频率	代码中的用途	性质
5 ms	摄像头距离/角度 PID 输出	源码调度配置
200 Hz	IMU/Mahony 采样频率宏	算法配置值
100 ms	新摄像头显示快照周期	Camera_New_DISPLAY_INTERV AL_MS
200 ms	结果最大允许年龄宏	Camera_New_RESULT_MAX_A GE_MS
3000 ms	目标丢失保持时间宏	Camera_New_TARGET_HOLD_ MS

5.3 摄像头多核流水线

CPU0 启动摄像头和流水线会话，DMA 完成中断发布图像槽位并切换下一块 DMA 目的地；CPU1 worker 获取最新帧，计算曝光统计、调用 Camera_New_ProcessFrame()，发布结果邮箱和显示快照，再释放处理槽位。CPU0 在跟随循环中读取稳定结果，控制车辆并周期性刷新显示。该结构将实时采集、识别计算和显示访问分开，减少在 DMA 中断中复制整帧图像的需要。

5.4 参数与 Flash 存储

Flash.c 对车辆路径点、科目参数、图像视觉和控制参数进行保存/读取。工程中已经加入旧参数兼容和小误差 P 参数的校验逻辑。

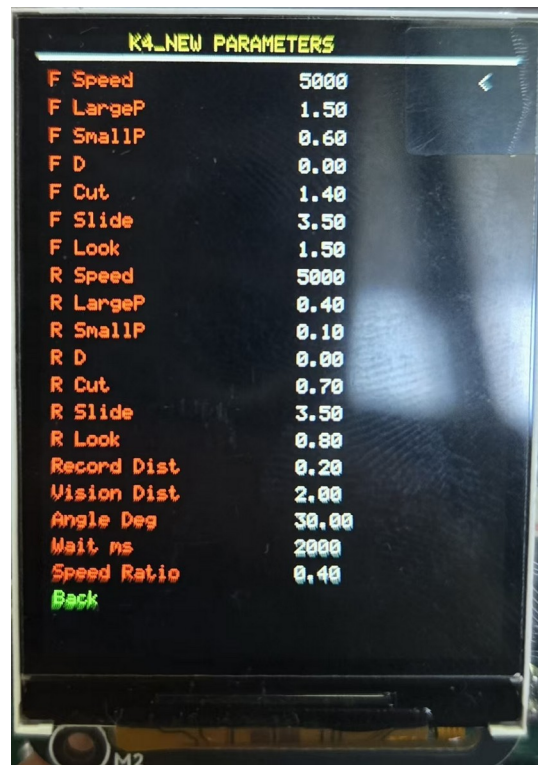


图 5.1

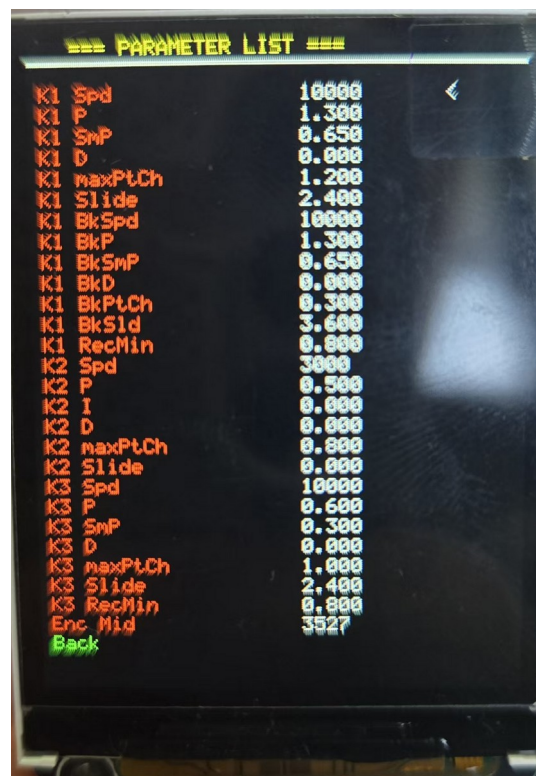


图 5.2

5.5 模型车主要技术参数

当前可由源码核实的内容与需要实物/实测补充的内容列于表 5.3。

表 5.3 模型车主要技术参数与待测数据

参数项目	源码可核实内容	最终填写值/备注
长×宽×高	源码未提供	长：120cm，宽：70cm，高：60cm（不算杆子）
电池总容量	北京科宇原装 T 型卡丁车电池	300W（预估）
摄像头	MT9V03X，源码识别 1 路	1 个
左右轮编码器	TIM4、TIM2，两路轮速编码器	2 个、逐飞科技增量式编码器
转向绝对值编码器	SPI4，一路	1 个、逐飞科技角度编码器
IMU	imu963ra 接口，一路	1 个

第六章 关键算法设计

6.1 摄像头图像处理与双圆识别

Camera_New_ProcessFrame() 的处理链路包括灰度统计、Otsu 阈值计算、二值化、连通域/洪泛填充、候选区域筛选、圆形度和填充率评价、双圆配对以及时序滤波。源码对候选数量、面积、宽高比、填充率、圆形度、周围白色比例、半径相似度、双圆间距比例和对角线斜率设置了明确宏。

表 6.1 双圆识别算法约束

算法约束	源码配置值	说明
最大候选数	32	CAMERA_NEW_MAX_CANDIDATES

算法约束	源码配置值	说明
候选面积范围	8~2500	像素，需结合图像尺寸复核
最小宽高比	0.50	候选区域筛选
最小填充率/圆弧度	0.40 / 0.35	候选区域筛选
周边白色比例	0.60	排除不符合目标背景的区域
半径相似度下限	0.55	双圆配对
双圆间距比例	2.50~20.0	双圆配对
对角线斜率范围	0.20~5.00	双圆连线筛选
最小配对置信度	0.45	结果质量门限

6.2 相机标定、跟踪与误差模型

源码设置相机标定最多采集 30 个有效样本，去除前后各 3 个样本后计算稳定参考值；标定内容包括双圆中心、间距、大小不对称度和板线角度。跟踪阶段对中心、间距和灰度模型进行低通或限幅更新，最多允许连续丢失 6 帧后进入完全丢失状态。

跟随误差包括横向误差、表观距离误差、大小不对称度、透视误差和转向误差。横向误差以图像中心归一化；距离误差由参考双圆间距与当前间距之比构造；透视误差使用当前大小不对称度减去标定值；转向误差将横向与透视项加权组合。权重和视场角配置需要在实车测试中复核。

6.3 曝光与灰度统计

新跟随路径使用 8 条居中采样行、行间距 5 像素遍历图像宽度，计算平均灰度和饱和像素比例。目标平均灰度为 90，灰度大于等于 245 视为饱和；当饱和比例达到 8% 时优先降低曝光。曝光候选值限制在 3~600，更新间隔为每 3 帧一次，写入失败时不提交新的当前曝光值。以上是源码控制逻辑，不等于已经完成环境覆盖测试。

6.4 电机速度与摄像头控制

左右轮分别使用 PID_Controller，参数独立保存（虽然有区分左右轮 PID，但实际使用中为相同值）。电机 PID 实现了积分抗饱和、积分输出限幅和目标速度为零时的最小启动占空比处理。摄像头距离和角度 PID 在当前配置下积分、微分参数为 0，实际配置为 P 控制；输出分别受电机占空比和转向角边界限制。

motor_set() 根据转向角计算左右轮差速，并对电机输出和转向电机输出设置斜率限制；摄像头新跟随转向控制还设置了死区、最小占空比、最大占空比和每 5 ms 的最大变化量，用于减少转向突变。

6.5 姿态、里程计与位置积分

IMU 初始化阶段连续采样 1000 次估计陀螺仪和加速度计零偏，IMU_Get() 在后续数据中扣除零偏。MahonyAHRSupdate() 使用陀螺仪和加速度计更新四元数并提取 Z 轴偏航角。Position.c 使用编码器里程增量和相邻时刻航向变化的中间角度进行二维位置积分，形式为 $x += ds \cdot \cos(\theta_{mid})$ 、 $y += ds \cdot \sin(\theta_{mid})$ 。

6.6 路径滤波与轨迹回放

SG_Filter.c 对路径点使用 11 点窗口的 Savitzky-Golay 系数，并跳过特殊点和间距不均匀的区域，以避免改变停止、倒车或科目边界。Trace.c 根据回放路径的目标点计算航向误差；对科目一、三、四可使用弧长前视选择器，前视距离按前进/倒车方向和科目配置区分，特殊点不被跨越，目标选择不会覆盖 Replay_Path.Index 的进度状态。

6.7 回放 PID 的可变 P 策略

轨迹回放中，源码将 5° 作为小误差边界、 15° 作为大误差边界，在小误差 P 和大误差 P 之间使用 smoothstep 平滑过渡。科目一前进、科目一倒车、科目三/四分别使用对应小 P 与大 P 参数，积分项在这些回放入口中置零；科目二保持独立参数路径。最终报告应补充各参数的标定过程和曲线。

第七章 参数标定与测试方案

7.1 标定顺序

建议按“机械中位与方向—编码器符号—IMU 零偏—电机速度环—转向环—摄像头安装与灰度标定—相机跟随—路径记录与回放—整车联调”的顺序进行。每次只改变一类参数，并保存修改前后值、环境条件和测试结果，以便回退和复现。

7.2 源码配置参数记录表

表 7.1 源码配置参数记录

参数类别	当前源码值/状态
左右电机 KP	2500.0f (PID.h)
摄像头角度 KP	160.0f; 近中心 90.0f
摄像头距离 KP	6000.0f
转向角上限	$\pm 30.0^\circ$
曝光目标灰度	90
曝光饱和阈值/比例	245 / 8%
回放前视距离	1.5 m; 倒车 0.8 m; 科目三/四 1.5 m
SG 滤波窗口	11 点; 起止保护点 15

第八章 问题分析与改进方向

8.1 可能的工程风险

表 8.1 工程风险与验证计划

风险	已有机制	仍需验证/改进
强光或过曝	自适应曝光、饱和比例判断、	室内外、阴晴变化和反光地面

风险	已有机制	仍需验证/改进
	曝光限幅	测试
目标短时丢失	最多 6 帧临时丢失状态、控制保持/停止逻辑	不同速度下的恢复时间和误跟随
DMA 与图像处理冲突	多槽位、所有权状态、非阻塞锁	高负载下丢帧与流水线故障计数
轮径/转向零位误差	独立轮速 PID、绝对值编码器中值、位置积分	机械复测和长距离漂移
电机突变和打滑	输出斜率限制、差速控制、速度闭环	不同电压和地面摩擦条件
参数版本混乱	Flash 保存/读取和菜单调参	导出参数清单并绑定软件版本

8.2 后续改进

后续可从四个方面改进：一是建立自动化参数扫描和可视化标定工具；二是将识别置信度、曝光、处理耗时和控制误差同步记录，形成可回放数据集；三是针对赛道任务将状态机、停止点和倒车逻辑独立建模；四是对多核流水线进行长时间压力测试，明确最坏情况下的帧延迟、槽位占用和故障恢复策略。

第九章 总结

本方案以 TC387QP 为核心，形成了从传感器采集、车辆状态估计、视觉识别到运动控制的完整软件闭环。源码已经体现出较清晰的模块划分：底层设备驱动与车辆状态相互独立，控制算法按电机、摄像头和轨迹回放分工，多核摄像头路径使用槽位和邮箱管理数据生命周期，便于后续扩展和调试。

目前本稿不对实车性能作结论。完成正式报告还需要补齐队伍信息、赛题流程、机械和电路图、标定过程、测试原始数据、性能曲线、问题复盘以及本届竞赛要求的格式字段。

参考文献

[1] Seekfree_TC387_Opensource_Library.zip: TC387 开源工程及其源码、测试脚本和工程文档。

[2] 卓晴. 全国大学生智能汽车竞赛技术报告撰写要求[EB/OL]. CSDN, 网页链接:
[https://blog.csdn.net/zhuoqingjoking97298/article/details/163552155。](https://blog.csdn.net/zhuoqingjoking97298/article/details/163552155)

[3] 全国大学生智能汽车竞赛组委会. 技术报告内容、评判标准、书写格式、提交方法说明[R].

附录 A 电路板详细设计原理图

A.1 主驱一体主控板详细原理图

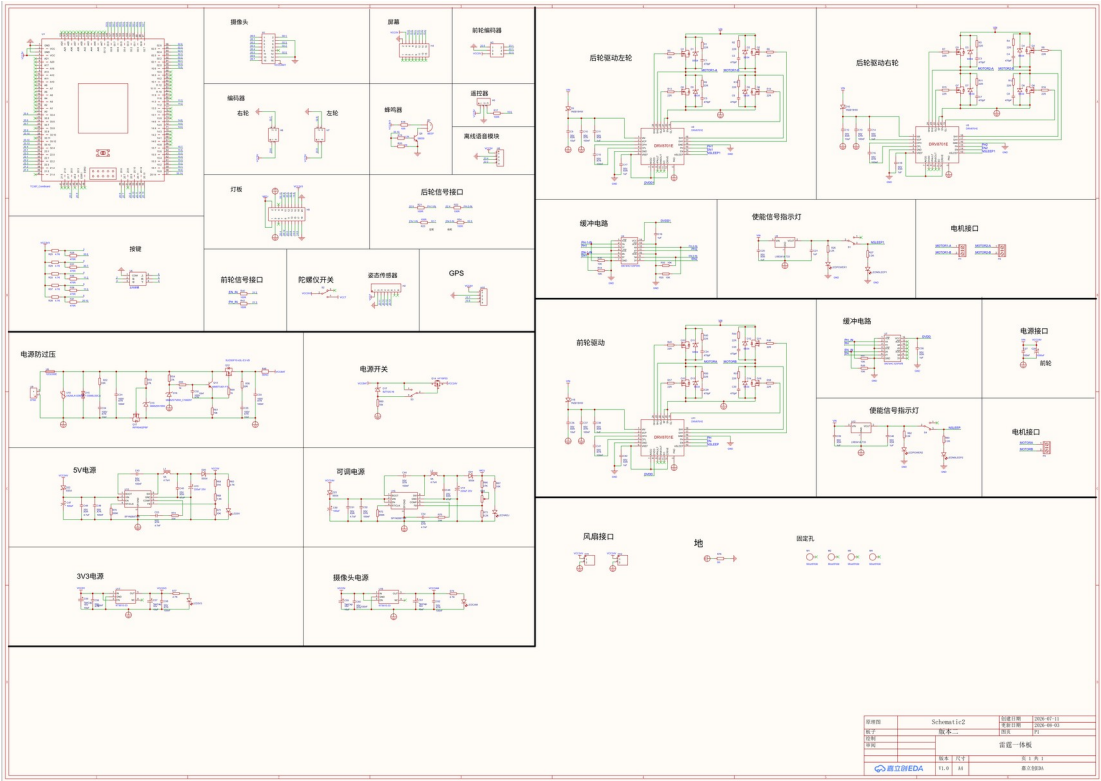


图 A-1 主驱一体主控板详细原理图

A.2 离线语音模块详细原理图

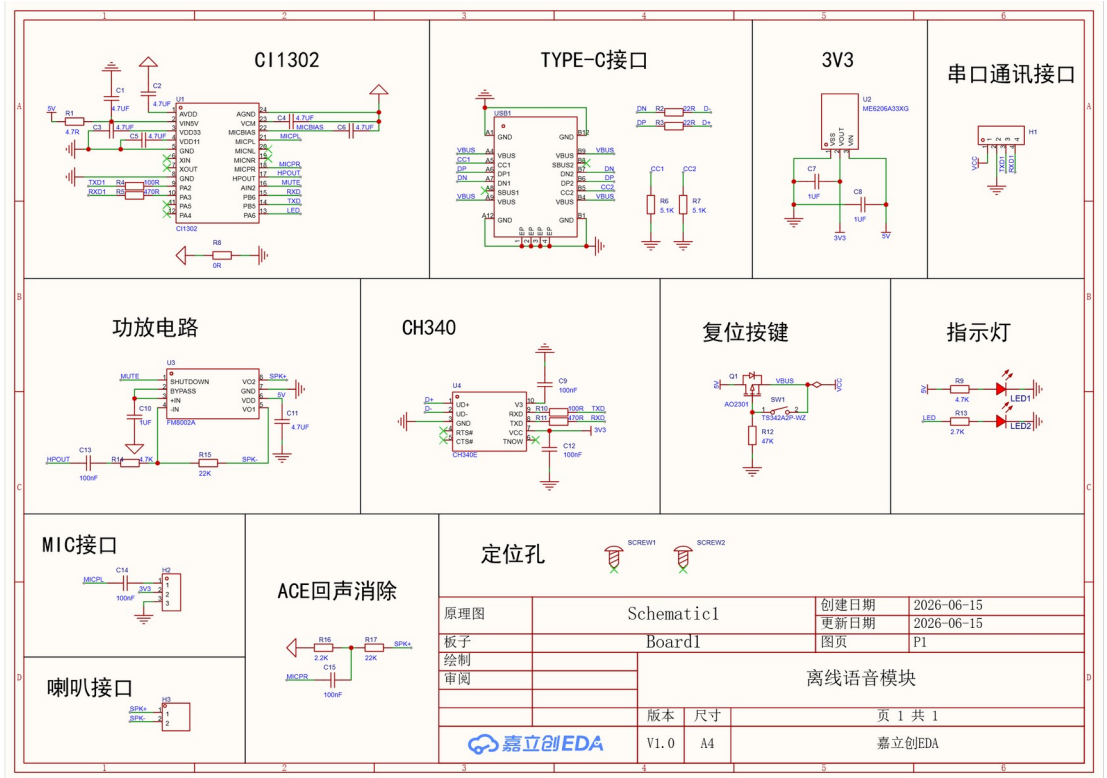


图 A-2 离线语音模块详细原理图

A.3 灯板驱动模块详细原理图

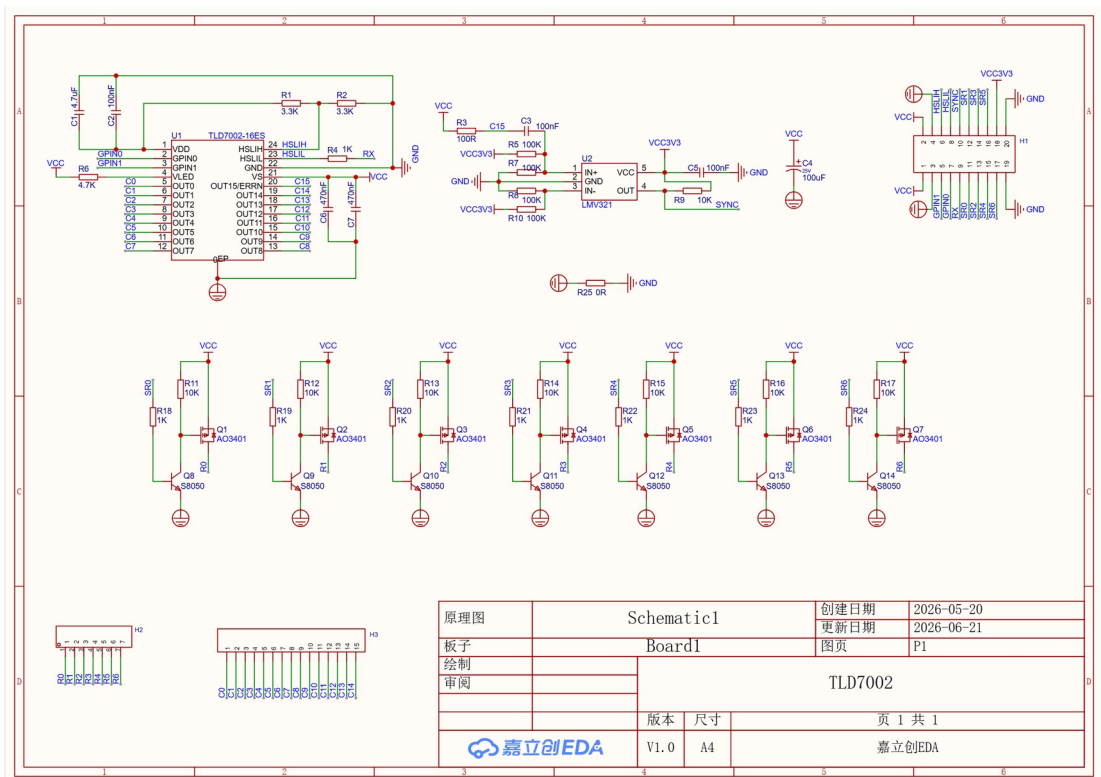


图 A-3 灯板驱动模块详细原理图

A.4 硅麦放大模块详细原理图

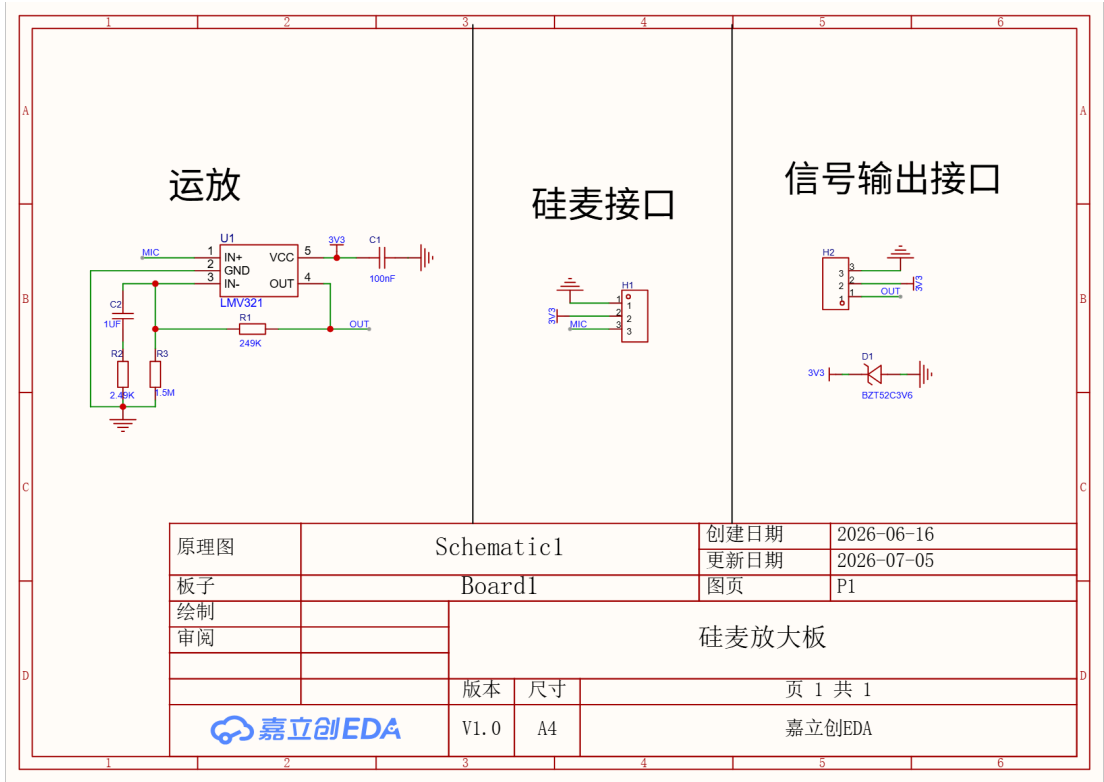


图 A-4 硅麦放大模块详细原理图

附录 B 程序源代码与电子附件说明

B.1 工程目录

表 B-1 程序源代码电子附件清单

电子附件目录	内容
code/Car_Device	摄像头、IMU、编码器、电机、Flash、显示、语音和通信驱动
code/Car_arithmetic	PID、位置、路径、科目和回放算法
code/user	CPU0~CPU3 入口、中断和调度配置
tests	源码契约和回归测试脚本
code/my_car	车辆主控系统，状态配置
code/menu	菜单
code/car_params	车辆参数

B.2 核心算法子程序

B.2.1 电机驱动、转向控制

```
#define MOTOR_SLEW_STEP_PER_CALL (33) // 每 5ms 最大变化 33 (占空比单位, 最大是 10000)

static int16 last_left_duty = 0;
static int16 last_right_duty = 0;
static int16 camera_new_last_steering_duty = 0;

/* 内部函数：带换向保护的斜率限制 */
```

```
static int16 motor_slew(int16 last, int16 target, int16 max_step)
{
    int16 step_target = target;
    int16 delta;

    /* 换向保护：如果符号相反，先回零 */
    if ((last > 0 && my_car.Speed < 0) || (last < 0 && my_car.Speed > 0))
    {
        step_target = 0;
    }

    delta = step_target - last;
    if (delta > max_step)    delta = max_step;
    else if (delta < -max_step) delta = -max_step;

    return last + delta;
}
```

```
static int16 steering_motor_slew(int16 last, int16 target, int16 max_step)
{
    int16 delta = target - last;

    if (delta > max_step)    delta = max_step;
    else if (delta < -max_step) delta = -max_step;

    return last + delta;
}
```

```
/*
* 新视觉模式的转向电机控制。
* 与原来的固定 5000/8000 占空比相比，这里按角度误差逐渐增加推力，
* 并限制每 5 ms 的占空比变化，避免转向电机突然抽动。
*/
```

```
static int16 camera_new_steering_motor_control(float expected_angle)
{
    float angle_error = expected_angle - my_car.duojie_angle;
    float absolute_error = fabsf(angle_error);
```

```

int16 target_duty = 0;

if (absolute_error > CAMERA_NEW_STEER_MOTOR_DEAD_ZONE)
{
    float duty_magnitude = CAMERA_NEW_STEER_MOTOR_MIN_DUTY +
        CAMERA_NEW_STEER_MOTOR_KP *
        (absolute_error - CAMERA_NEW_STEER_MOTOR_DEAD_ZONE);

    if (duty_magnitude > CAMERA_NEW_STEER_MOTOR_MAX_DUTY)
    {
        duty_magnitude = CAMERA_NEW_STEER_MOTOR_MAX_DUTY;
    }

    target_duty = (int16)duty_magnitude;
    if (angle_error < 0.0f)
    {
        target_duty = (int16)(-target_duty);
    }
}

camera_new_last_steering_duty = steering_motor_slew(
    camera_new_last_steering_duty,
    target_duty,
    CAMERA_NEW_STEER_MOTOR_SLEW_STEP);

return camera_new_last_steering_duty;
}

void motor_set(int duty, float angle)
{
    int left_duty, right_duty;    // 目标值（差速后）
    int left_apply, right_apply;  // 实际输出值（经斜率限制）

    // ===== 1. 差速计算（保持不变） =====

    if (angle > -DIFF_STEER_DEADZONE && angle < DIFF_STEER_DEADZONE)
    {
        left_duty = duty;

```

```
right_duty = duty;
}
else
{
    float angle_rad = fabsf(angle) * (PI / 180.0f);
    float tan_val = tanf(angle_rad);
    if (tan_val < 1e-4f) tan_val = 1e-4f;

    float R, ratio_outer, ratio_inner;
    if (Reverse_Start_Flag == 1)
    {
        R = car_L / tan_val;
        ratio_outer = (R + car_D_Back * 0.5f) / R;
        ratio_inner = (R - car_D_Back * 0.5f) / R;
    }
    else
    {
        R = car_L / tan_val;
        ratio_outer = (R + car_D * 0.5f) / R;
        ratio_inner = (R - car_D * 0.5f) / R;
    }

    int v_outer = (int)(duty * ratio_outer);
    int v_inner = (int)(duty * ratio_inner);

    if (angle > DIFF_STEER_DEADZONE)
    {
        left_duty = v_inner;
        right_duty = v_outer;
    }
    else
    {
        left_duty = v_outer;
        right_duty = v_inner;
    }
}
```

```

// ===== 2. 处理停止请求（软停止） =====
if (Stop_Flag == 1)
{
    // 目标设为 0，后续通过斜率限制逐步减到 0
    left_duty = 0;
    right_duty = 0;
    // 转向电机直接停止（保持原有逻辑，也可以加软停，但暂不变）
    gpio_set_level(DIR1_motor_duoji, 0);
    pwm_set_duty(PWM1_motor_duoji, 0);
    camera_new_last_steering_duty = 0;
}
else
{
    // 新视觉模式使用比例加斜坡控制，其他模式保留原来的控制。
    if (angle > MOTOR_DUOJI_ANGLE_MAX) angle = MOTOR_DUOJI_ANGLE_MAX;
    if (angle < -MOTOR_DUOJI_ANGLE_MAX) angle = -MOTOR_DUOJI_ANGLE_MAX;
    int motor_duoji_duty = 0;

    if (Camera_New_Following_IsControlActive())
    {
        motor_duoji_duty = camera_new_steering_motor_control(angle);
    }
    else
    {
        camera_new_last_steering_duty = 0;
        if (my_car.duoji_angle >= angle + 1.5f) motor_duoji_duty = -5000;
        if (my_car.duoji_angle <= angle - 1.5f) motor_duoji_duty = 5000;
        if (my_car.duoji_angle >= angle + 7.0f) motor_duoji_duty = -10 000;
        if (my_car.duoji_angle <= angle - 7.0f) motor_duoji_duty = 10 000;

        /* 新车模 */
        // if (my_car.duoji_angle >= angle + 1.5f) motor_duoji_duty = 5000;
        // if (my_car.duoji_angle <= angle - 1.5f) motor_duoji_duty = -5000;
        // if (my_car.duoji_angle >= angle + 7.0f) motor_duoji_duty = 10 000;
        // if (my_car.duoji_angle <= angle - 7.0f) motor_duoji_duty = -10 000;
    }

    // 输出转向

```

```

    if (motor_duoji_duty >= 0) {
        gpio_set_level(DIR1_motor_duoji, 0);
        pwm_set_duty(PWM1_motor_duoji, motor_duoji_duty);
    } else {
        gpio_set_level(DIR1_motor_duoji, 1);
        pwm_set_duty(PWM1_motor_duoji, -motor_duoji_duty);
    }
}

// ===== 3. 限幅（目标值） =====
if (left_duty > MAX_MOTOR_DUTY) left_duty = MAX_MOTOR_DUTY;
if (left_duty < -MAX_MOTOR_DUTY) left_duty = -MAX_MOTOR_DUTY;
if (right_duty > MAX_MOTOR_DUTY) right_duty = MAX_MOTOR_DUTY;
if (right_duty < -MAX_MOTOR_DUTY) right_duty = -MAX_MOTOR_DUTY;

// ===== 4. 斜率限制（软启停） =====
left_apply = motor_slew(last_left_duty, left_duty, MOTOR_SLEW_STEP_PER_CALL);
right_apply = motor_slew(last_right_duty, right_duty, MOTOR_SLEW_STEP_PER_CALL);

// 更新上次值
last_left_duty = left_apply;
last_right_duty = right_apply;

// ===== 5. 后轮驱动输出 =====

// 左轮

// /* 新车模 */
// left_apply = left_apply * 0.84;

if (left_apply >= 0) {
    gpio_set_level(DIR1_left, 0);
    pwm_set_duty(PWM1_left, left_apply);
} else {
    gpio_set_level(DIR1_left, 1);
    pwm_set_duty(PWM1_left, -left_apply);
}

// 右轮

```

```

// /* 新车模 */
// right_apply = - right_apply;

if (right_apply >= 0) {
    gpio_set_level(DIR1_right, 0);
    pwm_set_duty(PWM1_right, right_apply);
} else {
    gpio_set_level(DIR1_right, 1);
    pwm_set_duty(PWM1_right, -right_apply);
}
}

```

B.2.2 编码器获取、解算

```

/*
* 函数名：驱动电机编码器数值获取
* 备注：左右轮均为前进为正(包括：距离、速度)
*/
void encoder_motor_get()
{
    //1.获取左右轮速度

    /* 左轮 */
    int encoder_left = encoder_get_count(ENCODER_3);           // 获取编码器计数
    // int encoder_left = -encoder_get_count(ENCODER_3);         // 获取编码器计数（新车模）

    if(encoder_left * (2 * PI * car_R / sum_encoder_rad) > Ddistance / 2 || encoder_left * (2 * PI * car_R / sum_encoder_rad)
    < -Ddistance / 2)
    {
        my_car.speed_left = encoder_left * Ddistance * 1.21 / 0.005; //实际路程除以 5ms
    }else
    {
        my_car.speed_left = 0;
    }

    encoder_clear_count(ENCODER_3);                             // 清空编码器计数

    /* 右轮 */
    int encoder_right = -encoder_get_count(ENCODER_4);          // 获取编码器计数
    // int encoder_right = encoder_get_count(ENCODER_4);         // 获取编码器计数（新车模）

```

```

    if(encoder_right * (2 * PI * car_R / sum_encoder_rad) > Ddistance / 2 || encoder_right * (2 * PI * car_R /
sum_encoder_rad) < -Ddistance / 2)

    {

        my_car.speed_right = encoder_right * Ddistance * 1.21 / 0.005;//实际路程除以 5ms
    }else
    {

        my_car.speed_right = 0;

    }

    encoder_clear_count(ENCODER_4);                // 清空编码器计数

    // 车速

    my_car.Speed = (my_car.speed_left + my_car.speed_right)/2;

    //2.低通滤波

    /* 左轮 */

    double encoder_left_distance = 0;

    if(encoder_left * (2 * PI * car_R / sum_encoder_rad) > Ddistance / 2 || encoder_left * (2 * PI * car_R / sum_encoder_rad)
< -Ddistance / 2)

    {

        encoder_left_distance = encoder_left * Ddistance;

    }

    /* 右轮 */

    double encoder_right_distance = 0;

    if(encoder_right * (2 * PI * car_R / sum_encoder_rad) > Ddistance / 2 || encoder_right * (2 * PI * car_R /
sum_encoder_rad) < -Ddistance / 2)

    {

        encoder_right_distance = encoder_right * Ddistance;

    }

    //3.计算路程

    my_car.last_distance = my_car.distance;

    float dangle = my_car.YAW - my_car.last_YAW;//转向角度

    //  if(dangle > 0)//左转

    //  {

    //      my_car.distance += encoder_left_distance * 0.8 + encoder_right_distance * 0.2;

    //  }

    //  if(dangle < 0)//右转

    //  {

    //      my_car.distance += encoder_left_distance * 0.2 + encoder_right_distance * 0.8;

    //  }

    //  if(dangle == 0)

    //  {

```

```
// my_car.distance += encoder_left_distance * 0.5 + encoder_right_distance * 0.5;
// }

my_car.distance += (encoder_left_distance * 0.5 + encoder_right_distance * 0.5) * 1.21;
}
```

B.2.3 图像视觉处理

```
void Display_image_data(int red_flag, int yellow_flag, int display_image_flag, int dispaly_data_flag)
{
    /*
    * red_flag: 标红目标圆; yellow_flag: 标记两个圆心和目标中心;
    * display_image_flag: 显示图像; dispaly_data_flag: 显示几何与控制数据。
    * “dispaly”是已有接口拼写, 为避免改动调用关系而保留。
    */

    static int success_search_flag = 0;

    if (Image.Circle_sum != 2) {
        if(success_search_flag == 1)
        {
            success_search_flag = 0;

            ips200_clear();
        }

        ips200_displayimage03x((const uint8 *)Image.image, MT9V03X_W, MT9V03X_H);

        uint16_t line_height = 16; // 可根据 ips200_display_font 动态获取, 此处固定

        uint16_t text_y = MT9V03X_H + 2; // 图像底部留 2 像素空隙

        // 检查剩余空间是否足够显示两行
        if (text_y + 2 * line_height < ips200_height_max) {
            ips200_show_string(0, text_y, "cx_err:NULL!!!");

            ips200_show_string(0, text_y+line_height, "cy_err:NULL!!!");

            ips200_show_string(0, text_y+2*line_height, "R:NULL!!!");

            ips200_show_string(0, text_y+3*line_height, "dist:NULL!!!");

            ips200_show_string(0, text_y+4*line_height, "ang:NULL!!!");
        }

        return;
    }

    if(success_search_flag == 0)
    {
        success_search_flag = 1;
```

```

    ips200_clear();

}

// ===== 1. 生成基础图像（红色标记或普通二值图） =====
if (red_flag == 1) {
    image_with_red_mark(); // 生成包含红色标记的 rgb_buffer
} else {
    // 普通黑白二值图

    for (uint32_t r = 0; r < MT9V03X_H; r++) {
        for (uint32_t c = 0; c < MT9V03X_W; c++) {
            rgb_buffer[r][c] = (Image.image[r][c] == 255) ? RGB565_WHITE : RGB565_BLACK;
        }
    }
}

// ===== 2. 如果需要，绘制黄色标记（必须在红色生成之后） =====
if (yellow_flag == 1) {
    int x1 = (int)(Image_Control.x1+0.5f);
    int y1 = (int)(Image_Control.y1+0.5f);
    int x2 = (int)(Image_Control.x2+0.5f);
    int y2 = (int)(Image_Control.y2+0.5f);
    int cx = (int)(Image_Control.cx+0.5f);
    int cy = (int)(Image_Control.cy+0.5f);

    draw_point(x1,y1,RGB565_BLUE);
    draw_point(x2,y2,RGB565_BLUE);
    draw_point(cx,cy,RGB565_BLUE);
}

// ===== 3. 显示图像 =====
if (display_image_flag == 1) {
    ips200_show_rgb565_image(0, 0, (uint16_t*)rgb_buffer,
                            MT9V03X_W, MT9V03X_H,
                            MT9V03X_W, MT9V03X_H, 0);
}

// ===== 4. 显示数据（在图像下方，并检查空间） =====
if (display_data_flag == 1) {
    // 先获取当前字体高度（默认 8x16 => 16 像素）

```

```
uint16_t line_height = 16; // 可根据 ips200_display_font 动态获取，此处固定
uint16_t text_y = MT9V03X_H + 2; // 图像底部留 2 像素空隙

// 检查剩余空间是否足够显示两行
if (text_y + 2 * line_height < ips200_height_max) {

    ips200_show_string(0, text_y, "cx_err:");
    ips200_show_float(100, text_y, Image_Control.cx_error, 3, 3);
    ips200_show_string(0, text_y+line_height, "cy_err:");
    ips200_show_float(100, text_y+line_height, Image_Control.cy_error, 3, 3);

    ips200_show_string(0, text_y+2*line_height, "R:");
    ips200_show_float(100, text_y+2*line_height, Image_Control.Circle_R, 3, 3);
    ips200_show_string(0, text_y+3*line_height, "dist:");
    ips200_show_float(100, text_y+3*line_height, Image_Control.image_distance, 3, 3);
    ips200_show_string(0, text_y+4*line_height, "ang:");
    ips200_show_float(100, text_y+4*line_height, Image_Control.image_angle, 3, 3);

    ips200_show_string(0, text_y+7*line_height, "duoji:");
    ips200_show_float(100, text_y+7*line_height, raw_angle, 3, 3);
    ips200_show_string(0, text_y+8*line_height, "speed:");
    ips200_show_float(100, text_y+8*line_height, raw_speed, 3, 3);

}
}
```